

Reflection Template

Use Case	UC-2
Use Case Name	View User
Requirement IDs	FR2, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-4, CS-5
Implementation Order	Kotlin first -> Java second

Feature Relevance

Feature/s:

Null safety, extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, the extension function came up naturally when handling the “findById” call, instead of repeating the “orElseThrow” logic every time, it made sense to extract it. Null safety was not explicitly planned but is simply there by default in Kotlin, since the return type “UserResponse” cannot be null without explicitly marking it as “UserResponse?”.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	4
Compactness of logic	5	4
Perceived implementation effort	5	4

Justifications

Readability:

Both implementations are easy to follow. The extension function in Kotlin reads naturally “findByIdOrThrow” communicates what it does in a clear way. Java's “orElseThrow” inline is equally readable and well-known.

Compactness of logic:

Both are compact for this Use Case. The service logic is minimal in both languages. There a slight more effort for Kotlin because of Extension.kt.

Implementation effort:

Java felt slightly easier this time since the “orElseThrow” pattern is straightforward and required no extra setup. Kotlin needed an additional Extensions.kt file with the correct generic bounds (: Any), which took some extra thought to get right.

Observed structural differences

Java has no equivalent to Kotlin’s reusable extension function in Extensions.kt, “orElseThrow()” is repeated inline at every call site.

Also, Kotlin's return type UserResponse guarantees at compile time that getUser never returns null. Java has no such guarantee.

Both services use var for type inference, keeping local variable declarations equally concise.

Final comparative summary

Both implementations are nearly identical in readability and compactness this time. The main difference is the extension function: Kotlin allows pulling out repetitive repository logic into a reusable utility, Java has to repeat it inline. The effort was a bit for Kotlin due to the generic bounds issue in Extensions.kt.